

3-4 · DuckDB를 이용한 데이터 분석 · 실습 1.5H

SQL 집계·조인 분석 · 윈도우 함수와 CTE

파일끼리 JOIN 하고 Pandas 답과 대조

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① Parquet 와 CSV 를 적재 없이 곧바로 JOIN 한다
- ② SQL 결과와 Pandas 결과의 diff 가 0 인지 교차 검증한다
- ③ CTE + 윈도우 함수로 누적 매출·이동 평균을 낸다
- ④ WHERE 로 순위를 거르다 실패하고, QUALIFY·CTE 로 고친다

1~7단계를 sql_analysis.py 한 파일에 이어 붙여 단계마다 다시 돌립니다

일곱 단계 — 파일 넷이 있는 폴더에서

블록을 파일 끝에 붙이고 python3 sql_analysis.py — 앞 단계 출력이 함께 나오는 것이 정상

1. 상위 지출

GROUP BY
LIMIT 5

2. 파일 JOIN

parquet + csv
지역별 매출

3. 교차 검증

Pandas 와
diff = 0.0

4·5. CTE·윈도

누적 매출
지역별 상위 3

6·7. 프레임·편의

이동 평균 ma7
GROUP BY ALL

상위 지출 사용자와 파일 JOIN — 확인 수치

1단계 질의	WHERE event_type = 'buy' · GROUP BY user_id · ORDER BY spend DESC LIMIT 5
1단계 확인	1위 user_id 12844 · buys 6 · spend 44700
2단계 질의	'orders_step1.parquet' AS o JOIN 'customers.csv' AS c ON customer_id
2단계 확인	부산 276 · 77389000 / 경기 254 · 72923000
	서울 235 · 69007000 / 광주 250 · 68667000

price 결측 30건은 price * quantity 가 NULL — sum 은 건너뛰고 count(*) 는 센다

3단계 — Pandas 로 같은 질문, diff 를 뺀다

1 SQL 이 아니라 Pandas 로
orders_enriched.parquet 를 groupby('region') — orders.total(amount 합)

2 region 으로 merge
sql_result(2단계) 와 suffixes=('_sql', '_pd') · diff = total_sql - total_pd

3 diff 가 네 지역 모두 0.0
amount 와 price * quantity 가 결측을 똑같이 건너뛰었다

이 방법의 이름

교차 검증

0 이 아니면 어느 쪽이 결측을 다르게 다뤘는지부터 — 3-5 에서 도구를 옮길 때 쓰는 방법

CTE 로 두 문단 — 누적 매출과 지역별 상위 3

4단계 — 누적 매출

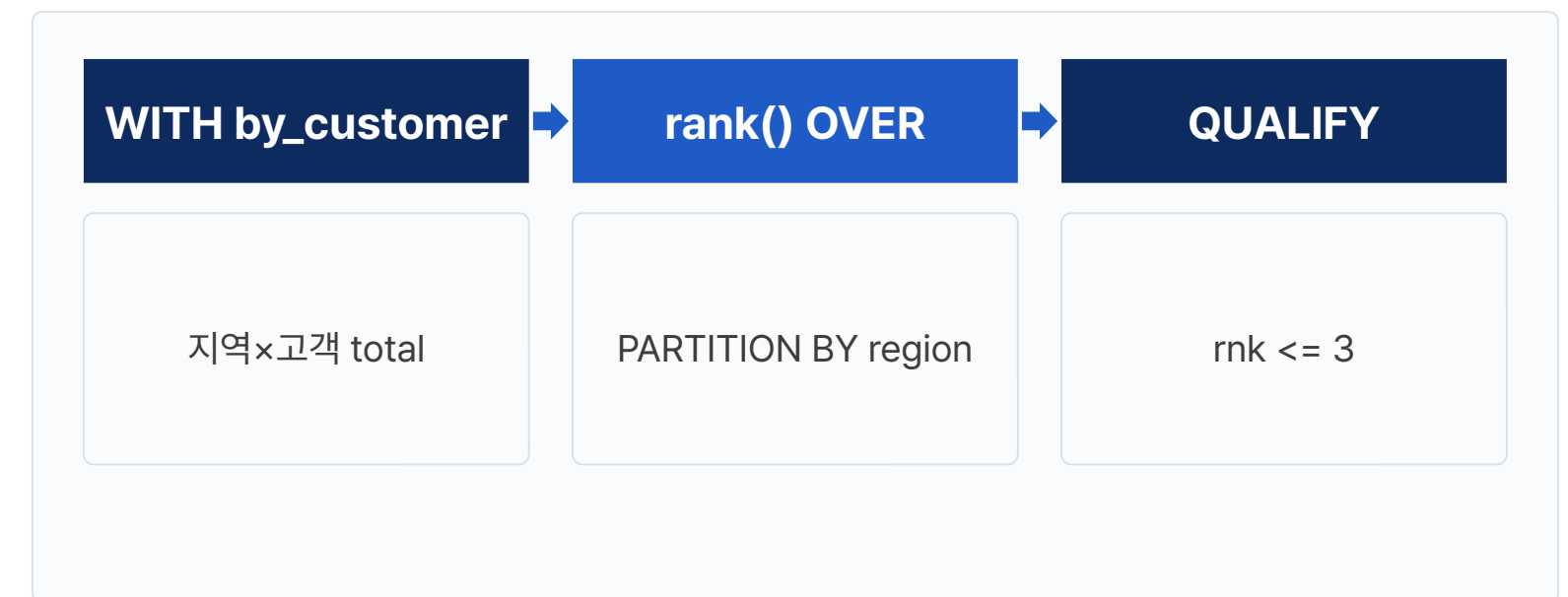
6월 1일 4064300 부터 단조 증가



VS

5단계 — 지역별 상위 3

12행 — 서울 1위 고객 19 · 4452000



윈도 함수를 WHERE 로 거르면 — Binder Error

실패 — WHERE $rnk \leq 3$

WHERE clause cannot contain window functions!

- WHERE 는 윈도 함수 계산 전에 행을 거른다
- 별칭은 보통 받아 주지만 윈도 함수만은 안 된다
- try 로 감싸 두어 파일은 멈추지 않는다

VS

해결 — QUALIFY 또는 CTE

두 표가 똑같이 12행(지역 4 × 3명)

- QUALIFY $rnk \leq 3$ — DuckDB 한 줄
- ranked AS (...) 로 감싸고 WHERE $rnk \leq 3$
- 동률이 있으면 3명을 넘을 수 있다

3-2 의 rolling(7) 을 SQL 로 — 앞쪽이 다름

Pandas rolling(7) — 3-2

창이 안 차면 NaN — 3-2 는 주문 데이터

- 창 7일이 다 차야 값이 나온다
- 앞 6일은 NaN
- 오늘 데이터(이벤트)와는 다르다

VS

SQL 윈도우 프레임 — 오늘

있는 만큼 평균 — 앞쪽도 값이 나온다

- ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
- 첫 줄 ma7 4064300.0 · 둘째 4186050.0
- 어느 쪽이 원하는 결과인지 정해야 한다

CTE 를 떼어 보는 습관 · DuckDB 편의 문법

- 1 daily 만 따로 실행**
CTE 안의 SELECT 만 LIMIT 3 — 6월 1~3일 3행이면 통과
- 2 GROUP BY ALL**
집계가 아닌 열로 알아서 묶는다 — 6월 1일 buy 818 · cart 2540 · view 13389
- 3 SELECT * EXCLUDE (event_id)**
몇 개만 빼고 전부 — 4월 3행
- 4 max(COLUMNS('^value\$'))**
열 이름 패턴으로 집계 — 9900

GROUP BY ALL 은 다른 DB 로 옮길 때 풀어 써야 한다

이 1.5시간이 끝날 때 확인할 것

- 1 1위 12844 · 44700, 지역 4행(부산 77389000 이 1위)
- 2 merged 의 diff 가 네 지역 모두 0.0 이다
- 3 cum_revenue 가 4064300 부터 단조 증가한다
- 4 WHERE rnk 는 Binder Error, QUALIFY·CTE 는 12행
- 5 ma7 앞쪽 며칠이 Pandas 와 다른 이유를 말할 수 있다

3-4 · 실습 1.5H 마무리

다음은 Pandas 연동 분석 (1H)

SQL 로 줄이고 DataFrame 으로 받아 등급을 매기고, 다시 SQL 로 질의하는 왕복을 roundtrip.py 에 이어 갑니다

✓ 눈대중 경계로 일부러 실패 → 분위수 경계

✓ FROM top — DataFrame 을 SQL 로

✓ SQL → Pandas → SQL 세그먼트 분석

✓ 줄이기 전 .df() 의 시간·메모리 실측

켜 둘 것

(.venv) 터미널 · events.parquet